RosterIQ

Memory-Driven Provider Roster Intelligence Agent

---

1. Problem Approach

Understanding the Problem

Healthcare payers process millions of provider roster submissions every month through a multi-stage operational pipeline. These submissions pass through several processing stages including pre-processing, mapping approval, ISF generation, DART generation, DART review, UI validation, and SPS load. Each roster file contains thousands of provider records that may succeed, fail, be skipped, or be rejected based on validation and compliance rules.

When failures occur, operations teams must manually investigate:

- which roster files failed
- which pipeline stages caused delays
- which provider organizations are responsible
- whether record-level quality issues are affecting market-level performance

The challenge becomes even more complex because operational data exists at **two different layers**:

File-Level Pipeline Data

Contains information about:

- pipeline stage durations
- record success/failure counts
- rejection and skip counts
- operational health flags

Market-Level Operational Metrics

Contains information about:

- monthly transaction success rates

- first-pass vs retry success
- overall market performance across states and clients

These two datasets represent **two different perspectives of operational health**.

Pipeline metrics explain **how files move through the system**, while market metrics explain **how well the system performs at the business level**.

Our goal was to build an AI agent capable of connecting these layers, diagnosing operational issues, and remembering past investigations to continuously improve its reasoning.

Solution Overview

We designed **RosterIQ**, an intelligent analytics agent that combines:

- LLM reasoning
- persistent memory
- structured data analysis
- visualization tools
- external knowledge retrieval

The system acts as an **autonomous operational analyst** that can:

- answer natural language questions about pipeline health
- diagnose stuck or failing roster operations
- correlate record-level quality issues with market success rates
- track historical investigations
- generate actionable operational insights

The most important component of the system is its **memory architecture**, which allows the agent to accumulate operational knowledge over time.

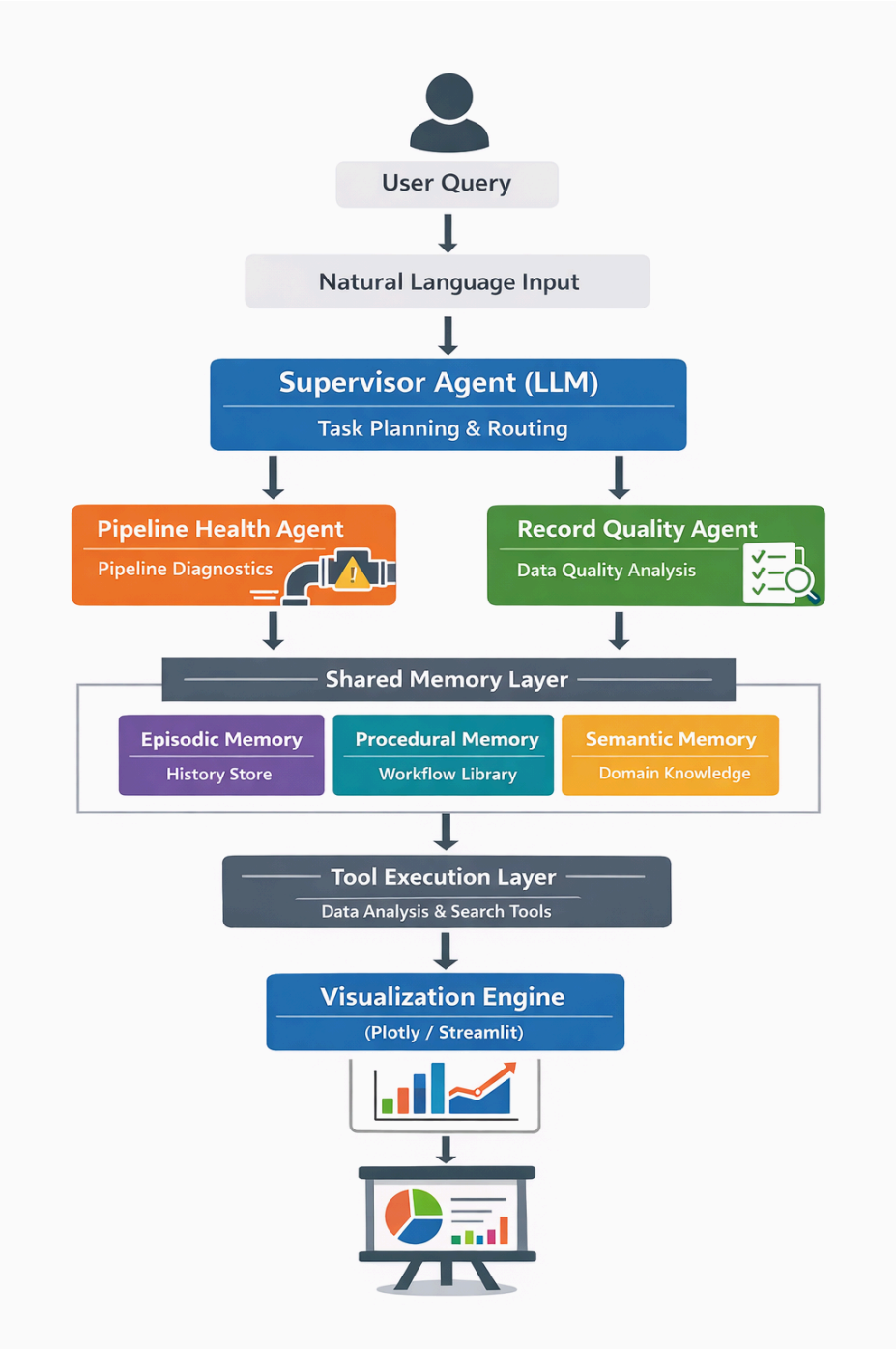
The architecture consists of three complementary memory layers:

1. Episodic Memory – remembers past investigations and findings
2. Procedural Memory – stores reusable diagnostic workflows
3. Semantic Memory – stores domain knowledge about provider roster pipelines

Together these memory systems allow the agent to behave like an experienced operations analyst rather than a stateless chatbot.

---

2. System Architecture



Data Flow

The agent processes user queries using a structured reasoning loop:

1. User submits a natural language question.
2. The Supervisor Agent interprets the query.
3. Relevant episodic memory is retrieved.
4. A diagnostic procedure may be executed.
5. Data analysis tools query the CSV datasets.
6. Web search may be triggered for external context.
7. Insights are visualized.
8. The interaction is stored in episodic memory.

This architecture allows the agent to **continuously learn from past interactions while producing structured operational insights.**

---

3. Memory Architecture Design

The most important requirement of the challenge was implementing **three distinct types of memory** that improve the agent's reasoning capability over time.

3.1 Episodic Memory

Purpose

Episodic memory records the **history of investigations performed by the agent**.

It allows the system to remember:

- what queries were previously asked
- what diagnostic actions were performed
- what findings were discovered
- how operational metrics changed over time

This enables the agent to provide **temporal awareness** across sessions.

Implementation

Episodic memory is implemented using a **vector database** (such as ChromaDB or FAISS).

Each stored memory contains:

- user query
- investigation summary
- detected issues
- timestamp
- relevant entities (state, organization, pipeline stage)

Example stored memory:

Query:
Investigate CA pipeline failures

Findings:
3 ROs stuck in DART_GENERATION

Timestamp:
2026-03-13

Retrieval Mechanism

When a new query is received:

1. The query is converted into an embedding.
2. Similar past investigations are retrieved.
3. The agent integrates this information into its reasoning.

Example output:

> “Last session you flagged three roster operations stuck in DART_GENERATION in CA. Two have since resolved, while one remains stuck and its failure count has increased.”

This allows the agent to track **changes across time**.

3.2 Procedural Memory

Purpose

Procedural memory stores **diagnostic workflows** that the agent can reuse.

Instead of reasoning from scratch every time, the agent executes predefined procedures that perform structured analysis.

Implemented Diagnostic Procedures

1. triage_stuck_ros

Identifies roster operations that are stuck in pipeline stages.

Steps:

1. Filter records where IS_STUCK = 1
2. Rank by duration in current stage
3. Prioritize files with red health flags
4. Generate escalation recommendations

---

2. record_quality_audit

Evaluates record-level quality for an organization or state.

Metric:

(Fail + Reject) / Total Records

Files below a success threshold are flagged for investigation.

---

3. market_health_report

Correlates market-level performance with pipeline failures.

This procedure joins:

- pipeline file data (CSV 1)
- market operational metrics (CSV 2)

The goal is to determine whether record-level quality issues are affecting market success rates.

---

4. retry_effectiveness_analysis

Evaluates whether reprocessing improves outcomes.

The procedure compares:

RUN_NO = 1
vs
RUN_NO > 1

This determines if retry pipelines actually increase success rates.

Procedural Learning

Procedures can be updated based on user feedback.

Example:

User instruction:

“Also include SKIP_REC_CNT in the quality audit.”

Updated procedure:

(Fail + Reject + Skip) / Total Records

The agent updates the stored workflow and confirms the change.

This enables **adaptive diagnostic learning**.

3.3 Semantic Memory

Purpose

Semantic memory stores **domain knowledge about provider roster pipelines**.

This knowledge allows the agent to interpret operational data and explain its reasoning.

Domain Knowledge Stored

Pipeline Stages

Stage	Description
Pre-Processing	Initial validation of roster files
Mapping Approval	Mapping of provider attributes
ISF Generation	Intermediate data structure creation
DART Generation	Data transformation
DART Review	Operational validation
UI Validation	Interface validation
SPS Load	Final system ingestion

Record Quality Metrics

Metric	Meaning
FAIL_REC_CNT	Processing failures
REJ_REC_CNT	Validation or compliance rejections
SKIP_REC_CNT	Records skipped due to business rules

Health Flags

Flag	Meaning
Green	Healthy pipeline stage
Yellow	Warning condition
Red	Critical operational issue

Example Reasoning

If rejection rates exceed a threshold:

REJ_REC_CNT / TOT_REC_CNT > 30%

The agent explains:

> “High rejection rates indicate validation failures rather than processing errors, suggesting the source data may not conform to expected schema standards.”

---

4. Tool Integration

Data Analysis Tools

Data analysis is performed using:

- Pandas
- DuckDB

Capabilities include:

- filtering and aggregation
- anomaly detection
- cross-table joins
- retry effectiveness analysis

Example anomaly detection rule:

Stage duration > 2 × historical average

Visualization Tools

The agent dynamically generates operational dashboards and charts.

Pipeline Health Heatmap

Displays:

Organization × Pipeline Stage

Colored by health flag.

Record Quality Breakdown

Stacked bar chart showing:

- Success
- Fail
- Skip
- Reject

Market Success Trend

Line chart showing:

Market SCS_PERCENT vs Month

Includes a 95% threshold reference line.

Retry Lift Chart

Compares:

First-iteration success vs post-retry success.

Web Search Integration

Web search allows the agent to retrieve external context.

Example use cases include:

- regulatory updates
- CMS rule changes
- Medicaid reporting requirements
- provider organization background information

This contextual information helps explain operational anomalies.

---

5. Innovations

Our solution introduces several innovations beyond the core requirements.

5.1 Cross-Layer Root Cause Tracing

The agent automatically traces operational issues across multiple layers.

Example chain:

Market Success Drop
→ Identify affected state
→ Identify failing roster files
→ Identify pipeline stage bottleneck
→ Identify responsible organization or source system

This provides a **complete causal explanation for operational failures**.

5.2 Adaptive Procedural Learning

Diagnostic procedures evolve based on user feedback.

Example improvement:

Original metric:

(Fail + Reject) / Total Records

Updated metric:

(Fail + Reject + Skip) / Total Records

This allows the system to accumulate operational expertise.

5.3 Memory-Driven Context Awareness

The agent uses episodic memory to detect patterns such as:

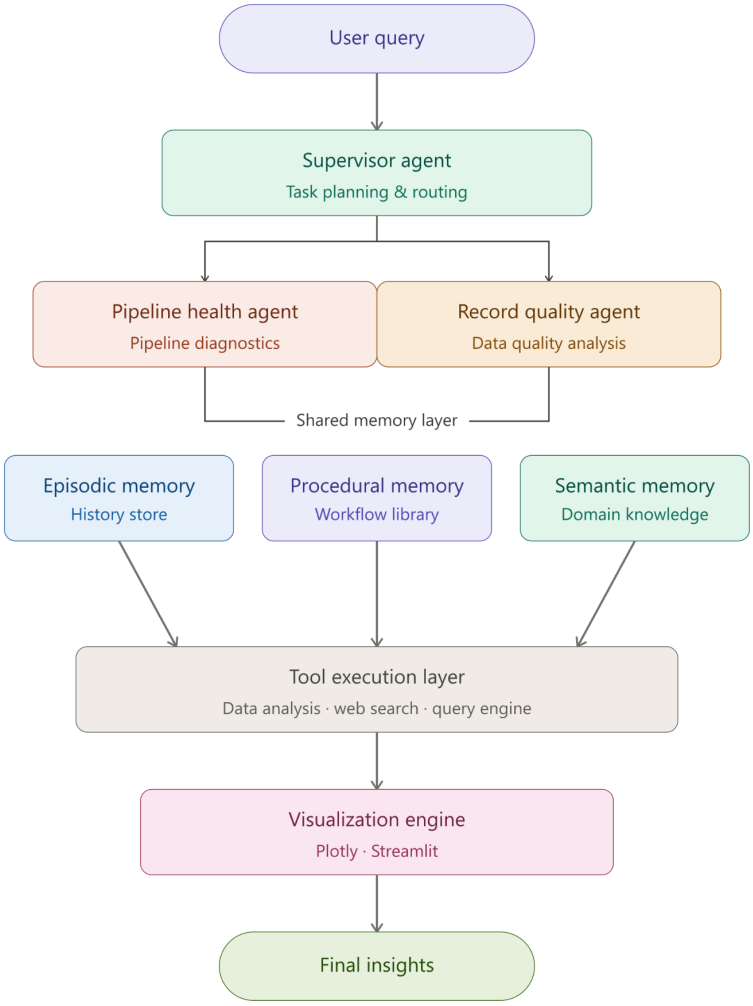
- recurring failures for specific organizations
- pipeline stages frequently becoming bottlenecks
- markets with unstable success rates

Example response:

> “Norton Hospitals' rejection rate has increased from 28% to 34% since the last investigation.”

5.4 Multi-Agent Architecture

To improve modularity and reasoning accuracy, we implemented a **multi-agent system** consisting of a Supervisor Agent and specialized domain agents.



Agent Responsibilities

Supervisor Agent

- interprets user queries
- decides which specialist agents to activate
- aggregates results
- produces final explanation

Pipeline Health Agent

Analyzes pipeline behavior.

Responsibilities include:

- detecting stuck roster operations
- identifying stage bottlenecks
- analyzing stage duration anomalies
- monitoring pipeline health flags

Record Quality Agent

Analyzes record-level quality issues.

Responsibilities include:

- computing rejection and failure percentages
- identifying problematic organizations
- correlating file-level failures with market performance
- evaluating retry effectiveness

Benefits of Multi-Agent Architecture

Improved reasoning accuracy

Specialized agents focus on specific domains.

Parallel investigation

Pipeline and data quality can be analyzed simultaneously.

Modular architecture

New agents can easily be added, such as:

- Compliance Monitoring Agent
- Predictive Failure Detection Agent
- Provider Organization Intelligence Agent

---

6. Challenges and Learnings

Designing Memory Systems

Separating episodic, procedural, and semantic memory required careful architectural planning.

Cross-Dataset Correlation

The two datasets operate at different aggregation levels.

We solved this by mapping:

State ↔ Market
Month ↔ Reporting period

Tool Orchestration

Ensuring that the agent invoked tools appropriately required structured reasoning prompts and workflow design.

Explainability

Operations teams require explanations rather than raw metrics.

Semantic memory allowed the agent to provide **interpretable operational insights.**

---

7. Future Improvements

Potential extensions include:

- proactive monitoring and alerting
- predictive failure detection
- automated root cause analysis
- reinforcement learning for workflow optimization.